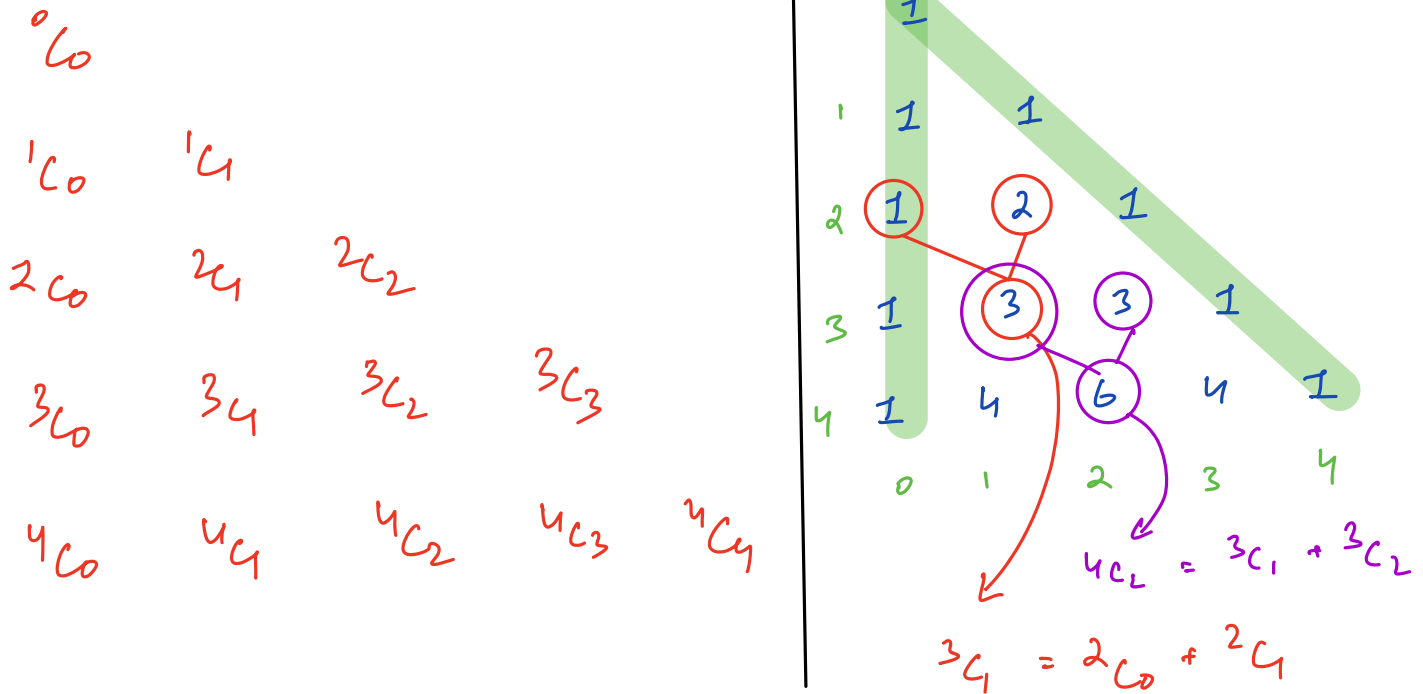


Lab Session on Prime Numbers & 2 Pointers

Question 1

Generate Pascal's Triangle for given N .

$N=4$



Brute force

if $iC_j \Rightarrow$ calculate it using $\frac{i!}{j! (i-j)!}$

TC to calculate $N! = O(N)$

$$\text{total TC} = O(N^2/2 \times N) = O(N^3)$$

Optimize

$$N_{C_0} = I \quad , \quad N_{C_N} = I$$

$$N_{C_R} = N^{-1} C_{R-1} + N^{-1} C_R$$

$$C(i)(j) = C(i-1)(j-1) + C(i-1)(j)$$

Code

```
def pascalTriangle(N) {
```

```
    nCr(n+1)(n+1) = {0}
```

```
    for(i=0 to n) {
```

```
        nCr(i)(0) = 1 , nCr(i)(i) = 1
```

```
        for(j=1 to i-1) {
```

```
            nCr(i)(j) = (nCr(i-1)(j-1) + nCr(i-1)(j)) % M
```

```
        }
```

```
    }
```

return n/r

}

$$TC = O(N^2)$$

$$SL = O(N^2)$$

Question 2

Given an array of size N , count divisors for each element.

$N=6$

$A = [1, 6, 3, 4, 2, 5]$

1	1	1	1	1	1
	2	3	2	2	5
	3		4		
	6				

ans: $[1, 4, 2, 3, 2, 2]$

Brute force

for $A[i]$, count the factors (divisors)

$$TC \text{ of factor count} = O(\sqrt{A[i]})$$

$$\text{Total TC} = O(N \times \sqrt{\max(A_i)})$$

code

```
int countFactors (a) {
    count = 0
    for (i = 1 ; i * i <= a ; i++) {
        if (a % i == 0) {
            if (i == a/i) count++
            else count += 2
        }
    }
    return count
}
```

```
int solve (A) {
    n = A.size()
    int (*) ans = new int[n]
    for (i = 0 to n-1) {
        ans[i] = countFactors (A[i])
    }
    return ans
}
```

$$TC = O(N \times \sqrt{\max(A_i)})$$

$$SC = O(1)$$

Optimize

count factors using prime factorization

$$6 = 2^1 \times 3^1$$

$$\begin{aligned}\text{factor count} &= (1+1) \times (1+1) \\ &= 2 \times 2 = 4\end{aligned}$$

$$48 = 2^4 \times 3^1$$

$$\begin{aligned}\text{factor count} &= (4+1) \times (1+1) \\ &= 5 \times 2 = 10\end{aligned}$$

$$30 = 2^1 \times 3^1 \times 5^1$$

$$\begin{aligned}\text{factor count} &= (1+1) \times (1+1) \times (1+1) \\ &= 2 \times 2 \times 2 = 8\end{aligned}$$

$$n = p_1^{a_1} \times p_2^{a_2} \times p_3^{a_3}$$

$$\text{no. of divisors} = (a_1+1) \times (a_2+1) \times (a_3+1)$$

Now to find p_1, p_2, p_3 a_1, a_2, a_3 for any number ?

Modify Sieve Algo

for each number, calculate smallest prime factor (spf).

	2	3	4	5	6	7	8	9	10
spf:	2	3	2	5	2	7	2	3	2

Algo:

1. initialise $spf[i] = i$ initially
2. iterate & if $spf[i] == i$ then
update spf of all multiples of i

code

int() set $spf(int\ a) \leq$ $\max(A[i])$

int() $spf = new\ int(a+1)$

for ($i = 1$ to a) {

$spf[i] = i$

}

```
for (i = 2; i * i <= a; i++) {
```

```
    if (spf(i) == i) { // i is prime
```

```
        for (j = i * i; j <= a; j += i) {
```

```
            if (spf(j) == j)
```

```
                spf(j) = i
```

```
        }
    }
```

```
return spf
```

```
}
```

$O(\max(A_i) \times \log \log (\max(A_i)))$

How to use spf to count divisors?

$48/2 = 24/2 = 12/2 = 6/2 = 3/3 = 1$ $2^4 \times 3^1$

$spf(48) = 2$ $spf(24) = 2$ $spf(12) = 2$ $spf(6) = 2$

$spf(3) = 3$

val = A[i]

ans = 1

while val > 1 :

 s = spf(val)

 count how many times s divides val

ans = (count+1)

Code

```
int solve ( int A ) {
```

```
    n = A.length
```

```
    ma = A[0] // max element in array
```

```
    for (i=1 to n-1) {
```

```
        ma = max ( ma, A[i] )
```

```
    }
```

```
    int spf = getSpf (ma); // create spf array
```

```
    int[] ans = new int[n]
```

```
    for (i=0 to n-1) {  $\longrightarrow O(N \times \log_2(A_i))$ 
```

```
        val = A[i]
```

```
        factors = 1
```

```
        while ( val > 1 ) {
```

```
            s = spf[val]
```

```
            cnt = 0
```

```
            while ( val % s == 0 ) {
```

$TC = O(\log_2(A_i))$

```

        val = val / s
        cnt++
    }
    factors *= (cnt + 1)
}
ans[i] = factors
}
return ans
}

```

$$\text{total TC} = O(\underbrace{\max(A[i]) * \log \log(\max(A[i]))}_{\downarrow \text{spf array}} + \underbrace{N \log_2(\max(A[i]))}_{\downarrow \text{main loop}})$$

$$\text{SC} = O(\max(A[i]))$$

Question-3 : Pairs with given sum

Given a sorted array A & integer K . find any pair

(i, j) s.t. $A[i] + A[j] = K$ and $i \neq j$
such that

$A = [-5, -2, 1, 8, 10, 12, 15]$ $K = 11$
 $1 + 10 = 11$

ans = (2, 4)

$A = [-3, 0, 1, 3, 6, 8, 11, 14, 18, 25]$ $K = 12$

iterate over all the pair and keep doing

1. Bruteforce : $TC = O(N^2)$ $SC = O(1)$

2. Hashing : $A[i] + A[j] = K$
 $A[j] = K - A[i]$

for, check if $K - A[i]$ is present

$TC = O(N)$ $SC = O(N)$

3. Binary Search (since data is sorted):

if, check if $K - A[i]$ is present using BS

$$TC = O(N \log N)$$

$$SC = O(1)$$

4. Two Pointers

→ where to place 2 pointers
→ how to update them

there are two ways

→ same corner
→ different corner

$$A = \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 \\ [-5 & -2 & 1 & 8 & 10 & 12 & 15] \\ i & j & & & & & \end{matrix}$$

$$K = 11$$

$$A[i] + A[j] = -5 - 2 = -7 < K$$

→ increase index i or j

Ambiguous !!

other ways (2nd ways) where i in left side and j in right side

$$A = \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 \\ [-5 & -2 & 1 & 8 & 10 & 12 & 15] \\ i & j & i & & j & j & j \end{matrix}$$

$$K = 11$$

$$A[i] + A[j] = -5 + 15 = 10 < K \Rightarrow \text{increase sum}$$

$$-2 + 15 = 13 > K \Rightarrow j-- \quad \text{Only option is } i++$$

$$-2 + 12 = 10 < K \Rightarrow i++$$

$$1 + 12 = 13 > K \Rightarrow j--$$

$$1 + 10 = 11 = K$$

$$\begin{aligned} -S + \text{largest of } A[i] &< K \\ -S + \text{any } A[i] &< K \end{aligned}$$

Code

$$i=0, \quad j=n-1$$

while ($i < j$) {

if ($A[i] + A[j] == K$)

return (i, j)

if ($A[i] + A[j] < K$) $i++$

else $j--$

}

return (-1, -1)

$$\begin{array}{ccccccc} [& 1 & 2 & 6 & 6 & 7 & 9 & 9 &] \\ & i & j & i & & j & j & j & \\ & & & & & & & & K=13 \end{array}$$

$$TC = O(N)$$

$$SC = O(1)$$

Question 4

find all pairs in sorted array where $\text{sum} = K$.

2 Parts :

1. Distinct array
2. Duplicates allowed

1. Distinct Array

$$A = [1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6 \quad 8] \quad K=10$$

$i \quad j \quad i \quad i \quad i \quad j \quad j$

$$1 + 8 = 9 < K \Rightarrow i++$$

$$2 + 8 = 10 = K \Rightarrow \text{cut}++, i++, j--$$

$$3 + 6 = 9 < K \Rightarrow i++$$

$$4 + 6 = 10 = K \Rightarrow \text{cut}++, j++, j--$$

$$\text{cut} = 2$$

code

$i=0, j=n-1, \text{cut}=0$

while ($i < j$) {

```
if (A[i] + A[j] == K) {
```

```
    cut++
```

```
    i++, j--
```

```
}
```

```
else if (A[i] + A[j] < K) {
```

```
    i++
```

```
}
```

```
else {
```

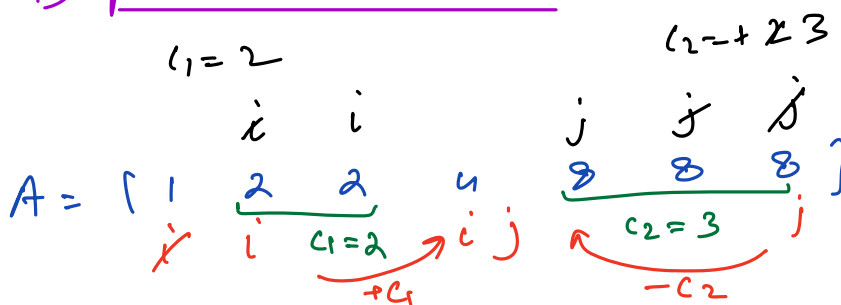
```
    j--
```

```
}
```

```
}
```

```
return cut
```

2. Duplicates Allowed



$K = 10$

$1 + 8 < 10 \Rightarrow i++$

$2 + 8 = 10 \Rightarrow \text{cut}++ = 6$

2 2's & 3 8's

$\Rightarrow 2 \times 3 = 6$ pairs

$$A = \begin{bmatrix} 1 & 2 & 5 & 5 & 5 & 5 & 8 \end{bmatrix} \quad K=10$$

$\overset{i}{\underset{\times}{1}} \quad \overset{j}{\underset{\times}{2}} \quad \overset{i}{\underset{\times}{5}} \quad \overset{j}{\underset{\times}{5}} \quad \overset{j}{\underset{\times}{5}} \quad \overset{j}{\underset{\times}{5}} \quad \overset{j}{\underset{\times}{8}}$

$$1 + 8 < 10 \Rightarrow i++$$

$$2 + 8 = 10 \Rightarrow C_1 = 1, C_2 = 1$$

$$cut++ = (1 \times 1) = 1$$

$$i++ = 1, j-- = 1$$

$$5 + 5 = 10 \Rightarrow cut++ = 4C_2 \rightarrow nC_2 = \frac{n \times (n-1)}{2}$$

$++ = 6$

$$nC_2 = \frac{4 \times 3}{2} = 6$$

Code

$i=0, j=n-1, cut=0$

while ($i < j$) {

if ($A[i] + A[j] < K$) $i++$

else if ($A[i] + A[j] > K$) $j--$

else { $A[i] + A[j] = K$

if ($A[i] == A[j]$) {

$C = j - i + 1$

$$\text{cut} += \frac{c * (c-1)}{2}$$

return cut

}

else { // $A[i] < A[j]$

$c_1 = 1$

while ($A[i] == A[i+1]$) {

$c_1++, i++$

}

$c_2 = 1$

while ($A[j] == A[j-1]$) {

$c_2++, j--$

}

$\text{cut} += c_1 * c_2 \Rightarrow \text{cut} = (\text{cut} + (c_1 * c_2) \% m) \% m$

$i++, j--$

}

}

}

return cut

$TC = O(N)$

$SC = O(1)$